1–25: Interface, Abstract Class & Polymorphism

1. Why does an interface not have a constructor?
Because an interface cannot be instantiated, so there is no interface object to initialize.

2. Why can an abstract class have a constructor?
Because its constructor initializes the common state of objects created by its subclasses.

3. Why can't an interface be instantiated?
Because an interface defines a contract, not a complete concrete implementation.

4. Why are interface variables public static final by default?
Because they are intended to be shared constants.

5. Why can an interface have static methods?
Because static methods belong to the interface itself, not to its objects.

6. Why were default methods introduced in Java 8?
To add new behavior to interfaces without breaking existing implementations.

7. Why can a default method have a body?
Because a default method provides an implementation that implementing classes can inherit.

8. What happens when two interfaces have the same default method?
The implementing class must resolve the ambiguity by overriding the method.

9. Why does a class method take priority over an interface default method?
Class implementation has higher priority than an inherited interface default.

10. Why can an interface extend multiple interfaces?
Because interfaces allow combining multiple contracts.

11. Can an interface extend a class?
No. An interface can extend only interfaces.

12. Can a class extend an interface?
No. A class implements an interface.

13. Can an interface implement another interface?
No. It extends another interface.

14. Why does Java support multiple interfaces but not multiple class inheritance?
Multiple class inheritance can create state and implementation conflicts.

15. What is the diamond problem in interfaces?
It occurs when multiple inheritance paths provide conflicting default implementations.

16. How is interface default-method ambiguity resolved?
The implementing class overrides the conflicting method.

17. What happens if a class method and interface default method have the same signature?
The class method takes priority.

18. Why is an interface reference useful?
It allows programming against an abstraction instead of a concrete class.

19. Why is inheritance called an IS-A relationship?
Because a subclass is a specialized type of its superclass.

20. Why is composition called a HAS-A relationship?
Because one object contains or uses another object.

21. Why is composition often preferred over inheritance?
It reduces coupling and allows behavior to be changed more easily.

22. Why can deep inheritance become difficult to maintain?
Changes in a parent class can affect many subclasses.

23. Why is method overloading compile-time polymorphism?
The compiler selects the method based on the arguments and method signature.

24. Why is method overriding runtime polymorphism?
The JVM selects the implementation according to the actual runtime object.

25. Why is dynamic method dispatch important?
It allows subclass-specific behavior through a parent or interface reference.

26–50: Overriding, Constructors & Initialization

26. Can a private method be overridden?
No. Private methods are not inherited.

27. Can a static method be overridden?
No. Static methods are hidden, not overridden.

28. Can a final method be overridden?
No. final prevents overriding.

29. Why can't a final class be extended?
Because final prevents inheritance.

30. Can a final reference refer to a mutable object?
Yes. The reference cannot change, but the object's state can change.

31. Why is upcasting useful?
It allows an object to be treated as a general parent type.

32. Why is downcasting risky?
The actual object may not be an instance of the target subclass.

33. Why use instanceof before downcasting?
To verify the object's runtime type before casting.

34. Why are fields not polymorphic like methods?
Field access is resolved using the reference type, not runtime dispatch.

35. What happens when a parent reference accesses a hidden field?
The field belonging to the reference's declared type is accessed.

36. Why can't an overriding method reduce visibility?
It would violate the accessibility promised by the parent class.

37. Can an overriding method increase visibility?
Yes.

38. What is a covariant return type?
An overriding method can return a subtype of the parent's return type.

39. Why can an overriding method throw fewer checked exceptions?
To maintain compatibility with the parent method's contract.

40. Why can't an overriding method throw broader checked exceptions?
It could break code that expects the parent method's narrower exception contract.

41. Why are constructors not inherited?
Constructors initialize a specific class and are not inherited members.

42. Can constructors be overloaded?
Yes, using different parameter lists.

43. Why can't a constructor be static?
Constructors create objects, while static members belong to the class.

44. Why can't a constructor be final?
Constructors cannot be overridden, so final has no purpose.

45. Can a constructor be abstract?
No. Constructors must contain concrete initialization logic.

46. Why must super() be called first?
The superclass part of the object must be initialized first.

47. What happens if we don't write super()?
The compiler inserts a no-argument super() if it is accessible.

48. What if the parent has no no-argument constructor?
The child must explicitly call an accessible parent constructor.

49. Can this() and super() both be directly used in one constructor?
No. Both must be the first statement, so only one can be used directly.

50. Why is constructor chaining useful?
It avoids duplicate initialization code.

51–75: Abstract Class, Static, Encapsulation

51. Can a constructor be private?
Yes. It can restrict object creation.

52. Can an abstract class contain final methods?
Yes. Those methods cannot be overridden.

53. Can an abstract class be final?
No. abstract requires subclassing, while final prevents it.

54. Can an abstract class have zero abstract methods?
Yes. It can be abstract simply to prevent direct instantiation.

55. Why use an abstract class instead of an interface?
When related classes need shared state, constructors, or common implementation.

56. Why use an interface instead of an abstract class?
When you need a contract that can be implemented by unrelated classes.

57. Can an interface contain private methods?
Yes, since Java 9.

58. Why were private interface methods introduced?
To reuse common implementation inside interface methods.

59. Can an interface contain protected methods?
No.

60. Can an interface contain instance variables?
No. Interface fields are implicitly static.

61. Why must interface fields be initialized?
Because they are implicitly final.

62. Why is static data shared by all objects?
Because only one class-level copy exists.

63. Why should mutable static variables be used carefully?
They create shared state and can cause coupling and thread-safety issues.

64. Can a static method directly access an instance variable?
No, because a static method does not have a specific object instance.

65. Why can an instance method access static members?
Because an instance method runs in the context of an object and can access class-level members.

66. Why can't an instance variable be accessed directly from a static context?
There may be no object from which to obtain that instance variable.

67. Can an abstract class have static methods?
Yes.

68. Can an abstract class have static variables?
Yes.

69. Why is encapsulation important?
It protects object state and controls how that state is modified.

70. Is using private fields alone enough for encapsulation?
No. Access must also be controlled properly.

71. Why can getters and setters still violate encapsulation?
They can expose internal mutable state without protection.

72. What is defensive copying?
Creating a separate copy to protect mutable internal data.

73. Why is immutability useful?
Immutable objects are easier to share and safer in concurrent programs.

74. Why is String immutable?
Once created, its state cannot be changed.

75. Why does immutability help thread safety?
Multiple threads cannot modify the object's state.

76–100: OOP Design & Object Concepts

76. What is an object invariant?
A condition that must remain true for a valid object.

77. Why should constructors establish invariants?
To prevent creation of invalid objects.

78. What is tight coupling?
Strong dependency between components.

79. What is loose coupling?
Components depend mainly on abstractions rather than concrete implementations.

80. Why is dependency injection useful?
It supplies dependencies externally and reduces tight coupling.

81. What is constructor injection?
Providing dependencies through a constructor.

82. Why is constructor injection suitable for mandatory dependencies?
The object cannot be created without them.

83. What is setter injection?
Providing dependencies through setter methods.

84. Why is setter injection useful for optional dependencies?
The dependency can be supplied or changed after construction.

85. Why is field injection often discouraged?
It hides dependencies and makes testing harder.

86. What is delegation?
One object forwards responsibility to another object.

87. Why is delegation useful?
It avoids unnecessary inheritance and separates responsibilities.

88. What is the Single Responsibility Principle?
A class should have one primary reason to change.

89. What is the Open/Closed Principle?
Software should be open for extension but closed for modification.

90. What is the Liskov Substitution Principle?
A subtype should be usable wherever its parent type is expected.

91. What is the Interface Segregation Principle?
Clients should not depend on methods they don't need.

92. What is the Dependency Inversion Principle?
High-level code should depend on abstractions rather than concrete details.

93. Why can inheritance break encapsulation?
A subclass can become dependent on parent implementation details.

94. Why should mutable collections sometimes be returned as copies?
To prevent callers from modifying internal state.

95. Why should fields usually be private?
To prevent uncontrolled external modification.

96. Why is programming to an interface useful?
Implementations can change without changing client code.

97. Why can polymorphism reduce if-else statements?
Different classes can encapsulate different behaviors behind one common type.

98. Why is an enum useful for fixed states?
It restricts values to a predefined set of constants.

99. Can an enum have a constructor?
Yes, but enum constructors are restricted and enum objects cannot be created using new.

100. Why is Object important in Java?
Every normal Java class ultimately inherits common methods from Object.

Part 2 — Collection Framework: 50 New Questions

101. Why does Collection extend Iterable?
To provide iterator-based traversal and support the enhanced for-loop.

102. Why does Map not extend Collection?
A Map stores key-value mappings rather than individual elements.

103. Why does Set reject duplicates?
Its contract requires unique elements.

104. Why does TreeSet need an ordering rule?
The tree needs ordering to place and search elements.

105. Why can TreeSet reject null with natural ordering?
Natural ordering requires comparisons between elements.

106. Why can HashSet contain null?
Its implementation can handle null specially.

107. Why does HashMap allow one null key?
It handles null as a special key.

108. Why can HashMap contain multiple null values?
Different keys can independently map to null.

109. Why are mutable HashMap keys dangerous?
Changing fields used by hashCode() can make the entry difficult to find.

110. Why must equals() and hashCode() agree?
Equal objects must have equal hash codes.

111. What happens if equals() is overridden but hashCode() is not?
Hash-based collections may behave incorrectly.

112. Why should hashCode use fields involved in equals?
To maintain the equality-hashCode contract.

113. Why is HashMap lookup usually O(1)?
Hashing generally identifies the bucket directly.

114. Why can HashMap lookup become slower?
Many hash collisions can increase bucket searching.

115. Why can HashMap buckets become trees?
To improve lookup under heavy collisions.

116. Why does HashMap resize?
When the load threshold is exceeded, it grows its table.

117. Why is load factor important?
It balances memory usage and collision/resizing frequency.

118. Why is initial capacity important?
A suitable initial capacity can reduce repeated resizing.

119. Why does LinkedHashMap preserve order?
It maintains links between entries.

120. Why is access-order LinkedHashMap useful for LRU caches?
Recently accessed entries can be moved toward one end.

121. Why does ArrayList have fast index access?
It uses an array internally.

122. Why is insertion at the beginning of ArrayList expensive?
Existing elements need to be shifted.

123. Why is removing from the middle of ArrayList O(n)?
Elements after the removed element need to shift.

124. Why is LinkedList index access slower?
It needs node traversal.

125. Why isn't LinkedList always better for insertion?
Finding the insertion location may require traversal.

126. Why is ArrayDeque often preferred over Stack?
It provides efficient deque/stack operations without Stack's legacy design.

127. Why is Stack considered legacy?
It is based on the older Vector design.

128. Why use Iterator.remove()?
It safely removes the current element through the iterator.

129. What is fail-fast behavior?
An iterator may detect unexpected structural modification and throw ConcurrentModificationException.

130. Why isn't fail-fast a thread-safety mechanism?
It is only best-effort detection.

131. What is weakly consistent iteration?
It can tolerate some concurrent changes without necessarily throwing an exception.

132. Why is CopyOnWriteArrayList good for read-heavy applications?
Readers can operate without locking while writes create a new array.

133. Why is CopyOnWriteArrayList expensive for frequent writes?
Writes copy the underlying array.

134. Why isn't PriorityQueue fully sorted during iteration?
Only the head is guaranteed to have the highest/lowest priority.

135. Why can PriorityQueue contain duplicates?
It provides ordering, not uniqueness.

136. Why is unmodifiableList() not necessarily immutable?
The original list can still be changed through another reference.

137. Why is Arrays.asList() fixed-size?
It is a list view backed by the original array.

138. Why can changing the array change Arrays.asList()?
The list is backed by that same array.

139. Why does List.of() reject null?
Its immutable collection implementation does not permit null elements.

140. Why are immutable collection factories useful?
They provide read-only collections with clear ownership.

141. Why use Comparator?
It provides an external ordering strategy.

142. What happens if Comparator returns 0 for two different objects in TreeSet?
TreeSet can treat them as duplicates.

143. Why does ConcurrentHashMap reject null?
It avoids ambiguity between absent mappings and null values.

144. Why is Hashtable considered legacy?
It uses an older synchronized collection design.

145. Why can Set uniqueness depend on Comparator?
Sorted sets use comparison results for ordering and duplicate determination.

146. Why is computeIfAbsent() useful?
It creates a value only when a key has no mapping.

147. Why is computeIfAbsent useful for grouping?
It can create a collection for each new key automatically.

148. Why should collection views be used carefully?
They can reflect changes in their backing collection.

149. Why can subList() changes affect the original list?
subList() returns a view backed by the original list.

150. Why can a collection's order change after modification?
Some collections don't guarantee insertion order, and internal restructuring can change traversal order.

Part 3 — Exception Handling: 25 New Questions

151. Why is Exception a class?
It represents throwable objects with state and stack information.

152. Why are checked exceptions compiler-enforced?
Java requires explicit handling or declaration of checked failures.

153. Why are RuntimeExceptions unchecked?
They generally represent programming errors or conditions not requiring mandatory handling.

154. Why is Error separate from Exception?
Errors generally represent serious JVM or system problems.

155. Why should we avoid catching Throwable?
It can catch serious Error conditions that applications usually shouldn't handle.

156. Why does catch order matter?
A broad catch before a narrow catch makes the narrow catch unreachable.

157. Why must checked exceptions be caught or declared?
Because Java's compiler enforces that rule.

158. Can RuntimeException be declared using throws?
Yes. throws can document that it may propagate.

159. Why is finally used?
It is commonly used for cleanup.

160. When might finally not execute?
For example, if the JVM terminates abruptly using System.exit().

161. Why use try-with-resources?
It automatically closes AutoCloseable resources.

162. Why does try-with-resources require AutoCloseable?
It needs a standard close() contract.

163. What are suppressed exceptions?
Exceptions that occur during resource closing while another exception is already being propagated.

164. Why use multiple catch blocks?
Different exception types may require different handling.

165. Why should exception messages be meaningful?
They help developers diagnose failures.

166. Why shouldn't exceptions be used for normal control flow?
It makes normal logic harder to understand.

167. Why can custom exceptions have multiple constructors?
To support different messages, causes, or contextual information.

168. Why create a checked custom exception?
To force callers to explicitly handle or declare a business condition.

169. Why create an unchecked custom exception?
When callers generally cannot or should not recover at every call site.

170. Why preserve an exception cause?
To retain the original root failure.

171. What is exception chaining?
Wrapping one exception inside another while preserving the original cause.

172. Why can logging and rethrowing be problematic?
The same exception may be logged repeatedly at multiple layers.

173. Why should resources be closed after failure?
Otherwise resources can leak and eventually be exhausted.

174. Why should catch blocks be specific?
Specific handling avoids hiding unrelated programming errors.

175. Why shouldn't exception messages contain sensitive information?
Messages may appear in logs or responses accessible to unintended users.

Part 4 — Multithreading: 25 New Questions

176. Why are threads lighter than processes?
Threads share process resources, while processes generally have separate memory spaces.

177. Why can multithreading improve responsiveness?
One thread can continue while another waits for I/O.

178. Why is shared mutable state dangerous?
Multiple threads can modify it in unpredictable orders.

179. What is a race condition?
The result depends on the timing of concurrent operations.

180. What is a data race?
Unsynchronized concurrent access to shared mutable data.

181. Why does synchronized provide mutual exclusion?
Only one thread can hold the same monitor lock at a time.

182. Why does synchronized provide visibility?
Lock release and subsequent acquisition create a happens-before relationship.

183. Why isn't volatile enough for i++?
volatile provides visibility but does not make compound operations atomic.

184. Why is i++ not atomic?
It involves read, increment, and write operations.

185. Why use AtomicInteger?
It provides atomic operations such as increment without a traditional synchronized block.

186. Why can deadlock occur?
Threads can wait indefinitely for locks held by each other.

187. How does lock ordering help prevent deadlock?
A consistent acquisition order prevents circular waiting.

188. Why is starvation different from deadlock?
Starvation means a thread doesn't get resources; deadlock means threads are circularly waiting.

189. What is livelock?
Threads remain active but repeatedly react without making progress.

190. Why must wait/notify use the correct monitor?
The thread must own that object's monitor.

191. Why does wait() release the lock?
It allows another thread to acquire the monitor and change the condition.

192. Why doesn't sleep() release the monitor?
Sleeping pauses the thread but doesn't release monitors it already owns.

193. Why can notify() be less predictable than notifyAll()?
It wakes only one waiting thread, which may not be the one able to proceed.

194. Why should wait() normally be inside a while loop?
The condition must be checked again after waking.

195. Why do spurious wakeups matter?
A thread can wake without the required condition becoming true.

196. Why use ExecutorService instead of creating many threads manually?
It manages thread creation, reuse, task execution, and shutdown.

197. Why does a fixed thread pool limit concurrency?
Only its configured number of worker threads execute tasks simultaneously.

198. Why can an unbounded task queue cause memory problems?
Tasks can accumulate faster than workers process them.

199. Why should ExecutorService be shut down?
Its worker threads can otherwise keep resources alive.

200. Why is Callable different from Runnable?
Callable can return a result and throw checked exceptions, while Runnable returns nothing.